

Truy xuất và Lưu trữ Dữ liệu với Python

Tập tin phẳng (CSV), Đa định dạng (Excel, JSON, HTML, PDF), SQLite & MongoDB

TS. Vũ Đức Minh

Khoa Khoa học Dữ liệu & Trí tuệ Nhân tạo – Đại học Kinh tế Quốc dân (NEU)

Tháng 9, 2026

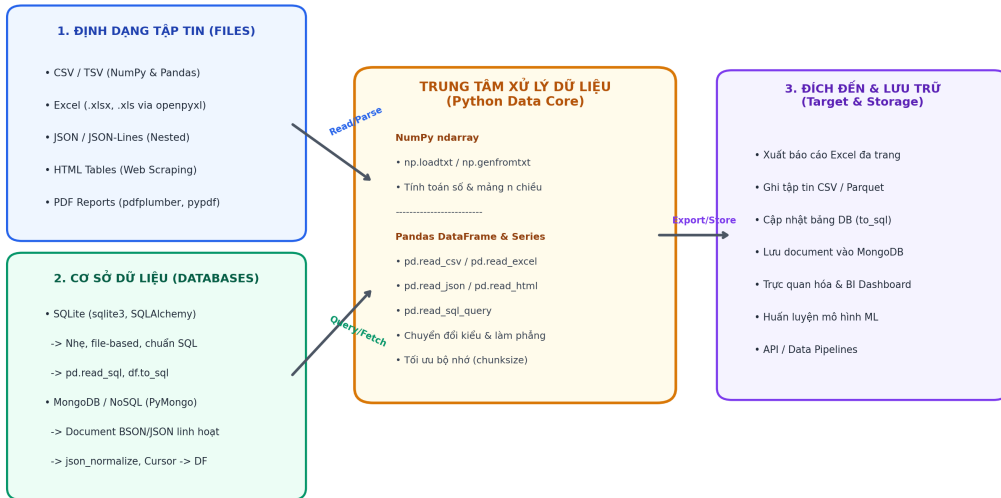
Mục tiêu bài học (CLO 5.4)

Sau khi hoàn thành bài học này, sinh viên có khả năng:

- ▶ **Đọc & ghi CSV:** Nắm vững `np.loadtxt`, `np.genfromtxt` và các tham số nâng cao của `pd.read_csv()` (`chunksize`, `encoding`, `parse_dates`).
- ▶ **Đa dạng định dạng tệp:** Làm việc với Excel đa trang tính (`openpyxl`), làm phẳng JSON lồng nhau (`json_normalize`), bóc tách bảng HTML và trích xuất PDF.
- ▶ **Cơ sở dữ liệu quan hệ:** Kết nối SQLite qua `sqlite3` & `SQLAlchemy`, thực thi truy vấn SQL và chuyển đổi hai chiều với Pandas (`read_sql_query`, `to_sql`).
- ▶ **Cơ sở dữ liệu NoSQL:** Kết nối MongoDB qua `pymongo`, thực hiện thao tác CRUD, chuẩn hóa BSON ObjectId sang DataFrame.
- ▶ **End-to-End Pipeline:** Thiết kế quy trình thu thập dữ liệu đa nguồn phục vụ báo cáo kinh doanh.

Tổng quan Hệ sinh thái Truy xuất & Lưu trữ Dữ liệu

HỆ SINH THÁI TRUY XUẤT VÀ LƯU TRỮ DỮ LIỆU TRONG PYTHON (Python Data Access & Storage Ecosystem)



Đọc dữ liệu CSV bằng NumPy

NumPy cung cấp hai hàm chính để làm việc với mảng số từ tệp văn bản:

`np.loadtxt()`

Dành cho dữ liệu số thuần túy, không có giá trị khuyết.

```
import numpy as np
# Đọc ma trận số thuần túy
arr = np.loadtxt("matrix.csv",
                 delimiter=";",
                 skiprows=1)
print(arr.shape)
```

`np.genfromtxt()`

Xử lý được giá trị khuyết và kiểu dữ liệu hỗn hợp.

```
# Thay thế giá trị thiếu bằng 0.0
arr_gen = np.genfromtxt(
    "numeric_sample.csv",
    delimiter=";",
    skip_header=1,
    filling_values=0.0
)
```

Xuất mảng ra CSV: `np.savetxt("out.csv", arr, delimiter=";", fmt="%0.2f")`.

Các tham số trọng yếu của Pandas read_csv()

```
import pandas as pd

df = pd.read_csv(
    "data/customers.csv",
    sep=";",
    index_col="customer_id",
    usecols=["customer_id", "full_name", "signup_date", "total_spent"],
    dtype={"customer_id": str},
    parse_dates=["signup_date"],
    na_values=["NA", "null", "-"],
    encoding="utf-8"
    # Ky tuphan cach cot (mac dinh la ',')
    # Cot lam chi muc Index
    # Tiet kiem RAM
    # Ep kieu ro rang tranh sai lech
    # Chuyen doi chuoai thanh kieu datetime
    # Cac ky tu quy uoc gia tri khuyet
    # Ho tro tieng Viet chuan
)
```

Mẹo xuất CSV tiếng Việt mở trên Excel

Dùng `df.to_csv("clean.csv", index=False, encoding="utf-8-sig")`. Cờ `utf-8-sig` bổ sung Byte Order Mark (BOM) giúp Excel tự nhận diện đúng tiếng Việt có dấu.

Xử lý tệp CSV dung lượng lớn với chunksize

Khi tệp CSV vượt quá dung lượng bộ nhớ RAM, nạp toàn bộ sẽ gây lỗi **MemoryError**.
Kỹ thuật **Chunking** giải quyết triệt để:

```
import pandas as pd

# Khởi tạo bộ lặp streaming đọc từng khối 100,000 dòng
chunk_iter = pd.read_csv("large_transactions.csv", chunksize=100000,
                        usecols=["store_id", "revenue"])

store_totals = {}
for i, chunk in enumerate(chunk_iter):
    # Xử lý từng khối đọc lặp trong bộ nhớ
    grouped = chunk.groupby("store_id")["revenue"].sum()
    for sid, rev in grouped.items():
        store_totals[sid] = store_totals.get(sid, 0) + rev
    print(f"-> Hoàn tất xử lý chunk #{i+1}")

print("Tổng doanh thu:", store_totals)
```

Làm việc với bảng tính Excel (openpyxl)

Đọc nhiều trang tính (Sheets)

```
# Đọc 1 sheet cụ thể
df = pd.read_excel("products.xlsx",
                  sheet_name="Products",
                  engine="openpyxl")

# Đọc TẤT CẢ các sheet (trả về dict)
all_sheets = pd.read_excel(
    "products.xlsx",
    sheet_name=None,
    engine="openpyxl"
)
```

Ghi nhiều Sheet vào 1 file

```
# Ghi multi-sheet bằng ExcelWriter
with pd.ExcelWriter("report.xlsx",
                  engine="openpyxl") as w:
    df_prod.to_excel(w,
                    sheet_name="Products",
                    index=False)
    df_kpi.to_excel(w,
                    sheet_name="Summary_KPI",
                    index=False)
```

Làm phẳng dữ liệu JSON phân cấp (json_normalize)

Dữ liệu JSON từ Web APIs thường có cấu trúc lồng nhau phức tạp (**Nested Objects & Arrays**).

```
import json, pandas as pd

with open("ecommerce_transactions.json", "r", encoding="utf-8") as f:
    orders = json.load(f)

# Làm phẳng mảng 'items' đồng thời mang theo các trường metadata cấp cha
df_flat = pd.json_normalize(
    orders,
    record_path=["items"],
    meta=[
        ["order_id", "order_date",
         "customer", "customer_id"],
        ["customer", "name"],
        ["customer", "city"]
    ],
    # Phân tách từng dòng theo từng item
    # Thuộc tính gốc của order
    # Trường con lồng nhau cấp 2
)
print(df_flat.head(3))
```

Bóc tách Bảng từ Web HTML & Báo cáo PDF

Web Scraping bảng với read_html

```
# Quét tất cả thẻ <table> từ URL/file
tables = pd.read_html(
    "financial_quotes.html",
    match="VN-Index" # Lọc theo từ khóa
)
df_stocks = tables[0]
print(df_stocks.head())
```

Trích xuất bảng PDF (pdfplumber)

```
import pdfplumber

with pdfplumber.open("invoice.pdf") as pdf:
    for page in pdf.pages:
        tables = page.extract_tables()
        for t in tables:
            df = pd.DataFrame(t[1:],
                               columns=t[0])

            print(df)
```

Mô hình Quan hệ (SQLite) vs Hướng tài liệu (MongoDB)

CƠ SỞ DỮ LIỆU QUAN HỆ (SQLITE)

Mô hình Bảng - Dòng - Cột (Schema cố định, ACID)

Bảng: Customers

customer_id	name	city	total_spent
101	Nguyễn Văn A	Hà Nội	2,450,000
102	Trần Thị B	Đà Nẵng	1,800,000
103	Lê Hoàng C	TP. HCM	5,120,000

Lệnh Python thường dùng:

- `conn = sqlite3.connect('company.db')`
- `df = pd.read_sql_query('SELECT * FROM Customers', conn)`
- `df.to_sql('Customers', conn, if_exists='append', index=False)`

CƠ SỞ DỮ LIỆU NOSQL (MONGODB)

Mô hình Document JSON/BSON (Schema động, lỏng nhau)

Collection: orders (Document mẫu)

```
{
  "_id": ObjectId("64a..."),
  "order_id": "ORD-9981",
  "customer": { "name": "Lê Hoàng C", "city": "TP. HCM" },
  "items": [
    { "sku": "P01", "qty": 2, "price": 1200000 },
    { "sku": "P05", "qty": 1, "price": 2720000 }
  ],
  "status": "completed"
}
```

Lệnh PyMongo thường dùng:

- `client = pymongo.MongoClient('mongodb://localhost:27017')`
- `docs = list(client.db.orders.find({'status': 'completed'}))`
- `df = pd.json_normalize(docs) # Làm phẳng dữ liệu l[ô]ng`

Cơ sở dữ liệu quan hệ SQLite & Pandas

```
import sqlite3, pandas as pd

conn = sqlite3.connect("data/company.db")

# 1. Truy van JOIN phuc tap truc tiep vao DataFrame
query = (
    "SELECT e.emp_id, e.full_name, d.dept_name, SUM(s.revenue) AS total_revenue "
    "FROM employees e "
    "JOIN departments d ON e.dept_id = d.dept_id "
    "JOIN monthly_sales s ON e.emp_id = s.emp_id "
    "GROUP BY e.emp_id, e.full_name, d.dept_name "
    "ORDER BY total_revenue DESC;"
)
df_sales = pd.read_sql_query(query, conn)

# 2. Luu tru nguoc tro lai co so du lieu SQLite
df_sales.to_sql("top_sales_summary", conn, if_exists="replace", index=False)
conn.close()
```

Cơ sở dữ liệu NoSQL MongoDB (pymongo)

```
from pymongo import MongoClient
import pandas as pd

# Ket noi cluster / local instance
client = MongoClient("mongodb://localhost:27017/")
db = client["retail_store_db"]
orders_col = db["customer_orders"]

# Truong hop 1: Chen document linh hoat
orders_col.insert_one({"order_id": "ORD-101", "items": ["P01", "P02"], "amount": 2500000})

# Truong hop 2: Loc du lieu va chuyen sang Pandas DataFrame
cursor = orders_col.find({"amount": {"$gte": 1000000}})
df_orders = pd.json_normalize(list(cursor))

# Xu ly khoa dac thu BSON ObjectId
if "_id" in df_orders.columns:
    df_orders["_id"] = df_orders["_id"].astype(str)
```

Bảng tổng kết tra cứu nhanh I/O trong Data Science

Định dạng	Hàm Đọc (Read)	Hàm Ghi (Write)	Thư viện chính
CSV / TSV	<code>pd.read_csv()</code>	<code>df.to_csv()</code>	Pandas (built-in)
Excel	<code>pd.read_excel()</code>	<code>df.to_excel()</code>	openpyxl
JSON	<code>pd.json_normalize()</code>	<code>df.to_json()</code>	json, Pandas
HTML	<code>pd.read_html()</code>	<code>df.to_html()</code>	lxml, bs4
PDF	<code>page.extract_tables()</code>	-	pdfplumber
SQLite	<code>pd.read_sql_query()</code>	<code>df.to_sql()</code>	sqlite3, SQLAlchemy
MongoDB	<code>col.find() -> DF</code>	<code>col.insert_many()</code>	pymongo

Nguyên tắc vàng về An toàn & Hiệu năng

1. Luôn dùng tham số hóa (?) chống SQL Injection. 2. Quản lý tài nguyên qua ngữ cảnh with. 3. Sử dụng usecols và chunksize khi làm việc với tập dữ liệu quy mô lớn.

Chúc các bạn thực hành hiệu quả!

Câu hỏi suy ngẫm cho buổi thực hành

- 1 Khi nào nên lưu trữ đơn hàng dưới dạng NoSQL Document thay vì bảng cơ sở dữ liệu quan hệ?
- 2 Làm thế nào để tự động hóa đường ống tải dữ liệu từ Web API (JSON) vào database SQLite chạy định kỳ mỗi đêm?

Liên hệ hỗ trợ: TS. Vũ Đức Minh (minhvd@neu.edu.vn)